2024 IEEE Intelligent Vehicles Symposium (IV)
June 2-5, 2024. Jeju Island, Korea

Band-S: Secure Band to Evade Cram Attacks to Queuing Disciplines for Ethernet-based In-Vehicle Networks

Ho Kang[1], Changjo Cho[1], Sol Ahn[1], Jangho Shin[2], Sanghoon Jeon[1], and Jong-Chan Kim[1,*]

2024 IEEE Intelligent Vehicle Symposium (IV) | 979-8-3503-4881-1/24/$31.00 ©2024 IEEE | DOI: 10.1109/IV55156.2024.10588406

***Abstract*—To satisfy the growing network bandwidth requirement of data-intensive automotive applications (e.g., autonomous driving), Ethernet is recently employed as the backbone of in-vehicle networks. Besides its high bandwidth, Ethernet-based systems require a much more complex software architecture than conventional networks (e.g., controller area network), leading to a system prone to unknown security vulnerabilities. This study reveals one of such attacks that obstruct safety-critical data flows between networked computers. In this attack, malicious applications generate garbage packets that are crammed into Linux transmit queues, blocking other important outgoing network packets. As a solution to this cram attack, we reserve a hidden transmit queue that can be used only by authorized applications through the trusted execution environment (TEE) of recent automotive application processors. For the evaluation, SOME/IP-based systems are implemented based on OP-TEE (an open source TEE), which successfully demonstrates our solution.**

I. Introduction

Recent automotive applications (e.g., autonomous driving) require a significant amount of network bandwidth, which cannot be satisfied by the conventional *controller area network* (CAN). Thus, high bandwidth *automotive Ethernet* is considered as the backbone of future in-vehicle networks [1]. In contrast to CAN-based systems, Ethernet requires a complex software architecture that includes protocol stacks (e.g., TCP and UDP) and device drivers (e.g., Ethernet controller driver). Due to the architectural complexity, there arise new attack surfaces for malicious applications [2]. To discover such vulnerabilities, we conducted a thorough analysis of the Linux Ethernet queuing system, which disclosed an attack scenario that can obstruct safety-critical in-vehicle data flows.

For example, let us assume the *zonal architecture* [3], where multiple *zonal gateways* are connected to a *central computer* through an Ethernet backbone network. Then a zonal gateway has a safety-critical outgoing data flow (e.g., camera images for emergency braking) to the central computer. If the data flow is unexpectedly delayed, it causes serious safety violations (e.g., collision without braking). In this regard, there are many studies about the denial-of-service (DoS) attack that overflows the network bandwidth by generating garbage packets from compromised *external* computers [4]. However, there are few studies that focus on the *internal* Linux Ethernet queuing system, which we thoroughly investigated. Through the investigation, we found that a single malicious application even without the root privilege can significantly delay the data flow by attacking the Linux Ethernet queuing system.

In more detail, network applications open sockets to send data to other networked computers. Each socket has a *socket queue* that temporarily holds the packets that are eventually delivered to the Ethernet *driver queue*. Between the two queues, there is another rarely known queuing system with multiple *transmit queues* that govern the Linux network queuing disciplines. Transmit queues support various packet scheduling algorithms from the socket queues (of multiple applications) to the driver queue (e.g., FIFO and Round Robin). Thus, if a malicious application cleverly generates garbage packets, the transmit queues will be crammed with useless garbage, and important safety-critical packets will be delayed and dropped. We name this attack method a *cram attack* [5].

As a solution to the cram attack, our approach is to make a *hidden* transmit queue, named *Band-S*, that can be safely used only by authorized applications. Since malicious applications do not know the existence of the hidden queue, a secure passage can be ensured for safety-critical packets even while the other transmit queues are crammed with garbage packets. In addition, we use the trusted execution environment (TEE), which is available in recent automotive application processors, to maintain the security even when the hidden queue is exposed. Since we assume ARM-based processors, ARM TrustZone is used with open source OP-TEE (open portable trusted execution environment) [6] that implements the GlobalPlatform API that is the standard programming interface for TEE services.

For the evaluation, we implemented SOME/IP-based data flow applications between two Nvidia Jetson platforms connected through Ethernet. The impact of the cram attack is measured by the packet transfer delay from the sender node to the receiver node. Our solution is implemented using the OP-TEE secure operating system and also by modifying a part of the Linux kernel that manipulates the transmit queues. Our experimental results show that the cram attack is a real threat to the in-vehicle network security, and our Band-S can be an effective mitigation method to it.

The contributions of this study can be summarized as:

- We present a new attack scenario that can significantly delay critical data flows between networked computing nodes in Ethernet-based automotive systems.
- We present a mitigation method to the above attack based on a hidden transmit queue that can be used only by authorized secure applications through TEE.

[1]Graduate School of Automobile and Mobility, Kookmin University, Korea. {rkdghrk12, whchangjo, solahn00, sh.jeon, jongchank}@kookmin.ac.kr
[2]Research & Development Division, Hyundai Motor Company, Hwasung, Korea.
*Jong-Chan Kim is the corresponding author.

Authorized licensed use limited to: Pontificia Universidade Catolica do Rio Grande do Sul (PUC/RS). Downloaded on March 26,2026 at 05:06:47 UTC from IEEE Xplore. Restrictions apply.

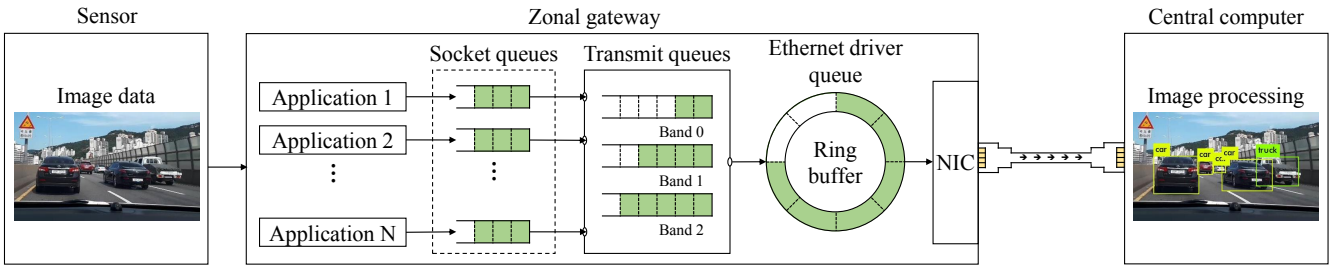


Fig. 1: Example data flow and Linux network queuing architecture.

II. Background

A. System Model

Fig. 1 shows our assumed automotive system with a critical data flow, where a camera sensor is attached to a zonal gateway that is also connected to a central computer through Ethernet. To make a data flow, the zonal gateway periodically transmits the received camera image to the central computer for further processing (e.g., object detection). In the zonal gateway, there are a number of network applications for various tasks, and one of them is transmitting the camera image. Each application creates its socket queue to transmit data using the standard Linux socket API. The packets in the socket queue must be delivered to the Ethernet driver queue (i.e., ring buffer) such that the network interface controller (NIC) can finally place the packets to the Ethernet bus. Between the socket queues and the ring buffer, there exist a rarely known queuing system, called *transmit queues*, for scheduling the packets from multiple socket queues to the ring buffer.

Linux *queuing discipline* (qdisc) manages the transmit queues for the traffic control of outgoing packets from concurrent network applications [7]. There are a number of transmit queue structures and scheduling algorithms, ranging from the simplest FIFO algorithm to more complex algorithms. Among them, the default one is *pfifo_fast* with three FIFO queues (Band 0, Band 1, and Band 2), where a priority-driven scheduling is enforced between them. Band 0 has the highest priority while Band 1 has the middle priority and Band 2 has the lowest priority. Thus, the packets in Band 2 cannot be transmitted while there are at least one packet in either Band 0 or Band 1. The Band can be selected when creating a socket considering each application's real-time requirement. Table I summarizes how socket queues are assigned to one of the three Bands depending on each socket's user-provided type of service (TOS) flag. As shown in the table, normal applications use TOS 0x0 that uses Band 1. In the case the application needs minimal delays, TOS 0x10 can be applied to use the highest priority Band 0 instead of the default Band 1. Although there are more complex qdiscs such as hierarchical token bucket (HTB) and stochastic fair queuing (SFQ), this study assumes the default pfifo_fast.

B. Trusted Execution Environment (TEE)

TEE is a secure area of a microprocessor that provides confidentiality and integrity of program executions by providing an isolated execution environment that cannot be compromised. We specifically use OP-TEE, which is an open source TEE implementation based on the ARM's TEE hardware (i.e., TrustZone). Fig. 2 shows the overall architecture of OP-TEE. OP-TEE provides a verified secure operating system that runs within TrustZone (i.e., *secure world*), operating alongside the normal operating system such as Linux (i.e., *normal world*). The separation between the two worlds is crucial for ensuring the security by TEE. OP-TEE ensures that only verified client applications (CAs) can interact with the secure world. Thus, verified CAs are allowed to make requests to trusted applications (TAs) running in the secure world. In OP-TEE-based systems, all the security-related tasks and data are offloaded to TAs in the secure world such that only the authorized applications can access confidential information and execute security-related operations such as key management services.

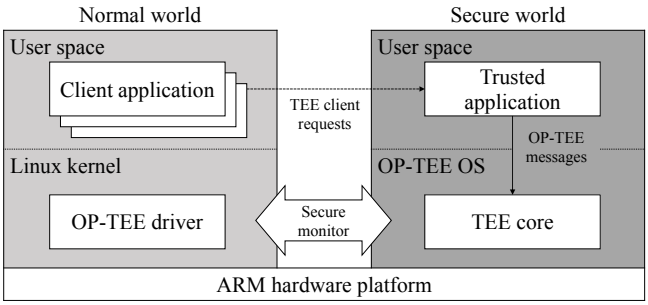


Fig. 2: OP-TEE architecture.

C. ISO 21434 Automotive Cybersecurity Standard

ISO 21434 [8], an automotive cybersecurity standard, provides the process for responding to cyber threats in automotive systems. This standard is becoming more and more crucial for addressing the security challenges posed by the increased connectivity of automotive systems. The advent of connected vehicles, which integrate various communication technologies such as Wi-Fi, Bluetooth, and 5G, has expanded the number of potential attack vectors for hackers. This increased connectivity complicates the task of securing automotive systems and responding to cyber threats. Consequently, these changes emphasize the need to follow ISO 21434 to respond to cyber threats.

Authorized licensed use limited to: Pontificia Universidade Catolica do Rio Grande do Sul (PUC/RS). Downloaded on March 26,2026 at 05:06:47 UTC from IEEE Xplore. Restrictions apply.

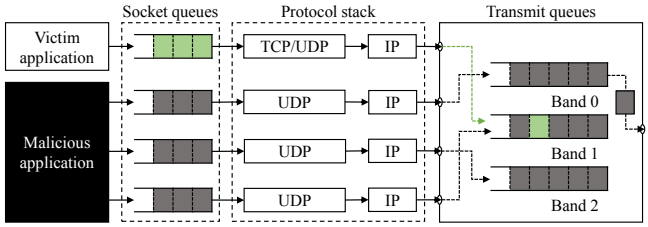


Fig. 3: Cram attack scenario.

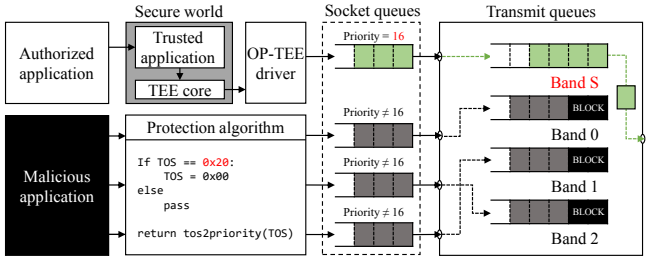


Fig. 4: Response to cram attack.

TABLE I: Relation between TOS and Bands in pfifo_fast.

TOS	Bits	Means	Priority	Band
0x0	0	Normal service	0	1
0x2	1	Minimize monetary cost (MMC)	1	2
0x4	2	Maximize reliability (MR)	0	1
0x6	3	MMC + MR	0	1
0x8	4	Maximize throughput (MT)	2	2
0xa	5	MMC + MT	2	2
0xc	6	MR + MT	2	2
0xe	7	MMC + MR + MT	2	2
0x10	8	Minimize delay (MD)	6	0
0x12	9	MMC + MD	6	0
0x14	10	MR + MD	6	0
0x16	11	MMC + MR + MD	6	0
0x18	12	MT + MD	4	1
0x1a	13	MMC + MT + MD	4	1
0x1c	14	MR + MT + MD	4	1
0x1e	15	MMC + MR + MT + MD	4	1
0x20	16	Secure service by Band-S	16	3

III. Attack Scenario

We assume the zonal gateway is broken into by a malicious application. Although we do not specifically assume a certain attack vector, recent automotive systems tend to have external connectivity through wireless networks and external storage devices that may expose the system to many cyber threats. Once the malicious application is somehow installed, it generates garbage packets after opening lots of sockets. Note that the malicious application does not need to know the IP address of the central computer since its attack point is inside the zonal gateway, neither on the central computer nor on the network bandwidth. Instead, the attack's purpose is to delay the delivery of the victim application's important packets by disturbing the packet flows *inside* the queuing architecture of the Linux network stack.

Fig. 3 illustrates the attack scenario that this study specifically deals with. The figure shows a victim application that generates packets through the TCP/UDP and IP stacks to Band 1, which is the default one for normal services among the three Bands. The malicious application generates UDP sockets, each of which is assigned to one of the three Bands. Assuming the pfifo_fast queuing discipline, the packets from Band 0 with the highest priority are fetched to the Ethernet driver queue following the priority-driven scheduling algorithm. If the malicious application continuously generates packets to Band 0, the victim application's packets in Band 1 have rare chance to go through to the Ethernet driver queue. For that, attackers can exploit the `setsockopt` [9] function to set the TOS flag to assign a socket to a particular Band with high priorities. In this way, malicious applications can generate garbage packets that effectively delay normal packets. We name this attack method a *cram attack*, meaning that garbage packets are crammed into the transmit queues to delay the delivery of victim packets.

IV. Response to Attack

Fig. 4 illustrates our proposed architecture to counter the cram attack. Our basic idea is to add a *hidden* transmit queue, which is named *Band-S*. For that, TOS 0x20 is newly added as in the last entry of table I such that applications can associate their sockets to Band-S. To provide security, Band-S has following properties. First, it is basically hidden from user-space applications and accessible only by authorized applications. There is no direct way of accessing Band-S since TOS 0x20 from usual applications is simply rejected by our kernel-level protection layer. Second, Band-S has the highest priority among the Bands, i.e., Band 0, Band 1, Band 2, and Band-S. Thus, if only authorized applications can use Band-S and malicious applications cannot access it, Band-S can be a secure passage from secure applications to the outer world even while the system is under the influence of the cram attack.

For authorized applications to use Band-S, we employ a hardware security mechanism known as TEE. Inside the secure world by TEE, we implement TAs that can handle the access to Band-S. To implement TAs, we use open source OP-TEE, and the operational flow of OP-TEE is a technical highlight of our approach. When the TA processes a request, it interacts with the TEE Core of the OP-TEE OS, subsequently issuing commands to the OP-TEE driver within the Linux kernel. Once an authorized application submits a request, the TA, operating within the secure confines of the OP-TEE OS, processes it. This involves utilizing the current process's file descriptors to track and manage the application's socket connections. By using these file descriptors, the TA can set socket priority to 16, ensuring that only legitimate data is transmitted through the Band-S.

In our security architecture, a fundamental design principle is that only authorized applications are permitted to request to

Authorized licensed use limited to: Pontificia Universidade Catolica do Rio Grande do Sul (PUC/RS). Downloaded on March 26,2026 at 05:06:47 UTC from IEEE Xplore. Restrictions apply.

the TA. This constraint is critical for maintaining the integrity and security of our system. It aims to prevent unauthorized or malicious entities from accessing the TA. To enforce this principle, there are authentication mechanisms that verify both the identity and integrity of applications before they are granted access to the TA. One of the authentication mechanisms is utilizing cryptographic techniques. The method of digital signatures to verify the identity of an application is the one [10]. Each application is required to present a certificate, signed by a trusted authority, confirming its authenticity. Additionally, the integrity of the application is ensured through hash checks, where the application's hash is compared against a known good hash to detect any tampering or corruption. In this process, public and private keys are used, playing a crucial role in identity verification. Digital signatures based on public key infrastructure (PKI) [11] offer a method to authenticate and confirm the legitimacy of applications.

V. Delay Analysis

A. General Analysis

This section analyzes how the cram attack impacts the image transmission delay. Data packets are transmitted from the zonal gateway to the central computer through (i) socket queues, (ii) protocol layer, (iii) transmit queues, (iv) ring buffer, and (v) physical network.

Socket queues. The delay experienced in the socket queue, denoted as d_{sock}, represents the time taken until the packet is extracted from the socket queue. Since socket queues are individual to each application, garbage packets cannot go into each other's socket queue, thus not having a direct impact.

Protocol layer. The raw data from socket queues undergo further processing through protocol stacks such as TCP/UDP and IP, where the raw data is split into packets and each protocol header is appended to the packets. This protocol processing time is denoted as $d_{protocol}$.

Transmit queues. After the network stack, the packets enter one of the transmit queues following the TOS flag. Although Band 1 is the default one for usual applications, we assume that Band 0 is used by our test applications since Band 1 tends to be totally blocked by garbage packets. The time taken for data to enter and exit a transmit queue is denoted as d_{band}.

Ring buffer. Following its exit from a transmit queue, the packet reaches the ring buffer in the Ethernet controller driver. The time taken for the packet to be transmitted from the ring buffer to the NIC is denoted as d_{ring}.

Physical network. The time for the packet to be physically transmitted to the central computer is denoted as $d_{network}$. Since our system architecture involves a singe connection between the two nodes, there is no additional network contention delay due to the absence of collisions typically associated with multiple network nodes.

After all, the total delay, denoted as d_{total}, is given by the sum of all the above delay components, as

$$d_{total} = d_{sock} + d_{protocol} + d_{band} + d_{ring} + d_{network}. \quad (1)$$

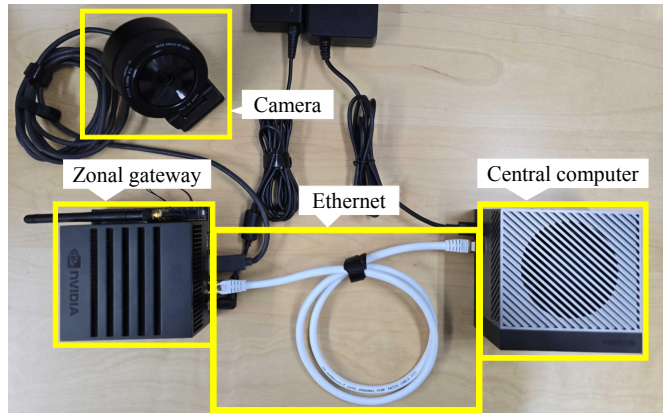


Fig. 5: Testbed implementation.

B. Comparison between TCP and UDP

Now, our focus is on the comparison of how the impacts are different under UDP and TCP protocols during the attack. UDP transmits packets in the socket buffer asynchronously, regardless of data losses during the transmission, due to its non-recovery nature. This transmission method is particularly beneficial to real-time applications where low latency is crucial but the delivery guarantee is not that important. In contrast, TCP transmits a packet and waits for an acknowledgment (ACK). The protocol does not send the next packets from the socket queue until the ACK is received. Additionally, TCP retransmits the packet if the ACK is not received within a specified deadline, ensuring a reliable transmission.

In the case of UDP, packet drops happen when transmit queues are full. In such scenarios, packets from socket queues that fail to enter the transmit queue can be dropped and lost. Additionally, the absence of an ACK system, coupled with asynchronous transmission, can result in d_{sock} becoming zero.

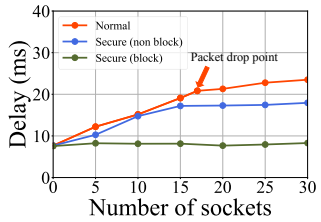
In the case of TCP, when the time from the moment data exits the socket to the receipt of an ACK, denoted as $(d_{total} - d_{sock})$, can exceed $1/FPS$ which represents the time interval between successive image frames captured by FPS based camera system which is represented by

$$d_{total} > \frac{1}{FPS} + d_{sock}. \quad (2)$$

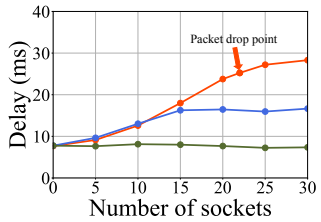
This leads to each frame progressively accumulating in the socket, causing d_{sock} to incrementally increase after every frame, ultimately resulting in a delay accumulation.

Simply using Band-S alone can reduce the d_{total} by driving the d_{band} to zero. However, if the packet in Band-S is periodic, there will be times when Band-S has no packet, potentially allowing garbage packets from Bands 0, 1, and 2 to be transferred to the ring buffer. This can lead to an escalation in d_{total} due to the increase of the d_{ring}. To prevent this, as shown in Fig. 4, activation of Band-S continuously blocks Bands 0, 1, and 2 to ensure the delay is not impacted by cram attacks.

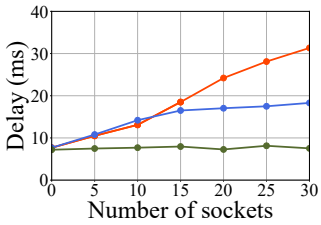
Authorized licensed use limited to: Pontificia Universidade Catolica do Rio Grande do Sul (PUC/RS). Downloaded on March 26,2026 at 05:06:47 UTC from IEEE Xplore. Restrictions apply.



(a) l = 500.

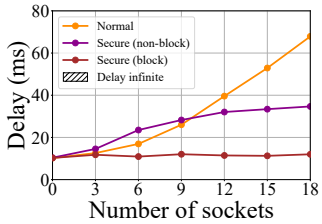


(b) l = 1000.

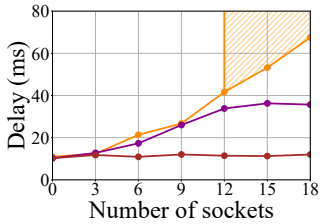


(c) l = 2000.

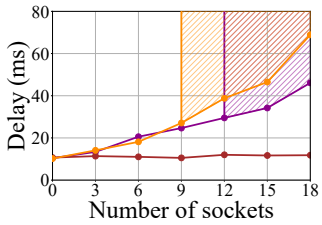
Fig. 6: UDP delay measurements.



(a) 15 FPS.



(b) 30 FPS.



(c) 60 FPS.

Fig. 7: TCP delay measurements.

VI. Experiments

A. Implementation

As in Fig. 5, we deployed the Nvidia Jetson AGX Orin as the central computer and the Nvidia Jetson AGX Xavier as the zonal gateway. The zonal gateway is equipped with the Razer Kiyo Pro USB 3.0 camera, capable of delivering output at the selected resolution of 640 x 480, reaching up to 60 FPS. Our software platform is based on Nvidia Ubuntu 20.04.6 LTS, featuring Linux Kernel 5.10.104 and JetPack 5.0.2. For integrating the GlobalPlatform API [12] using OP-TEE, we implemented the OP-TEE Trusted OS-Nvidia rel 35 and the OP-TEE Client API 4.0.0 library. Additionally, in our pursuit of a secure mechanism, we undertook the modification of the Linux kernel, a critical step in fortifying our system's security infrastructure.

B. Evaluation

We considered the number of garbage sockets as the intensity of the attack, plotting it on the x-axis, and measured the resulting image transmission delay, which we plotted on the y-axis. In Figs. 6 and 7, each graph displays three lines: **Normal line** corresponds to the scenario where image transmission data shares the same Band as the garbage data without any response to the attack. In **Secure (non-block) line**, it allows image data to flow through Band-S without blocking Band 0, 1, and 2. Since Band-S has a higher priority than the existing Bands, it is transmitted first. In **Secure (block) line**, as before, normal image data flows through Band-S, while the existing Bands are blocked by our designed security policy. In summary, these graphs demonstrate how the average delay of normal image data is affected by the three scenarios.

1) UDP Case: Fig. 6 illustrates, when using UDP protocol, how the three lines respond to cram attacks based on the queue length of the Band (l). For the experiments, we divided the Band length into three categories, 500, 1000, and 2000, while the ring buffer size is 1024 and the camera frame rate is 30 FPS.

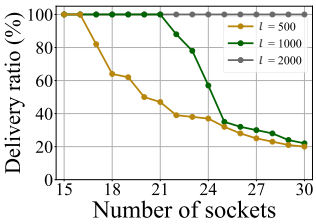


Fig. 8: Packet delivery ratio.

Normal line. In Fig. 6a, packet drops are observed when the intensity of the attack is 17. In Fig. 6b, the same is observed when the intensity of the attack is 22. However, packet drops are not observed in Fig. 6c. The reason for the different packet drop points is the varying lengths of the Band, resulting in different attack intensities required to fully utilize the Band, indicating that the Band of l = 2000 is not fully utilized even at the attack intensity of 30. After the packet drop point, the slope becomes less steep, indicating a further increase in delay. This happens because the number of garbage sockets contending with the socket for normal image data increases. As observed in Fig. 8, as the garbage socket increases, the probability of the Band being fully utilized also increases, leading to more frequent packet drops and a subsequent decrease in the delivery ratio.

Secure (non-block) line. Regardless of the Band length, Figs. 6a, 6b, and 6c all show the same upward slope, demonstrating that the delay stabilizes after an attack intensity reaches 15. Given that Band-S holds the highest priority, d_{band} does not influence this outcome. When Band-S is empty, packets from other Bands enter the ring buffer, leading to an increase in d_{total} in Eq. 1. This increase is due to d_{ring}, the delay until the entire ring buffer is transmitted.

2) TCP Case: Fig. 7 illustrates, when using TCP protocol, how the three lines respond to cram attack based on the camera FPS. For the experiment, we divided the FPS into three categories, 15, 30, and 60, while the Band length is 1000 and the ring buffer size is 1024.

Normal line. In Figs. 7b and 7c, we observe the delay accumulation at the attack intensities of 12 and 9, respectively. However, this is not observed in Fig. 7a. The reason for the difference in the starting point of delay accumulation is that, as the FPS increases, the delay according to attack intensity satisfies the condition in Eq. 2 earlier. In 15 FPS, the delay does not satisfy the condition even at the attack intensity of 18.

Secure (non-block) line. In Figs. 7a and 7b, the delay increases up to an attack intensity of 15, when the ring buffer becomes full. After this point, the delay remains constant as there is no reason that would increase d_{total}. In Fig. 7c, owing

Authorized licensed use limited to: Pontificia Universidade Catolica do Rio Grande do Sul (PUC/RS). Downloaded on March 26,2026 at 05:06:47 UTC from IEEE Xplore. Restrictions apply.

to the higher FPS, the delay satisfies the condition in Eq. 2 at the attack intensity of 12, indicating that despite the use of Band-S, the delay accumulation occurs.

3) For Both Cases: **Secure (block) line**. In both Figs. 6 and 7, the use of Band-S results in the blocking of Bands 0, 1, and 2, making d_{band} and d_{ring} unaffected by the cram attack. Consequently, it is observed that d_{total} remains constant regardless of the attack intensity.

VII. Related Work

The utilization of QoS methods to respond to DoS attacks has been a significant motivator for our study [13], [14]. This method encompasses dividing traffic flows into separate groups, assigning priorities to each group, and designating different bandwidth allocations for these groups. Previous studies have indeed explored the use of Linux qdisc as a QoS method for mitigating DoS attacks [15], [16]. They use a token bucket filter (TBF) or HTB that DoS data flows into the default Band, while legitimate data is prioritized in higher Bands, and DoS-occupied Bands receive minimal bandwidth, represented as a token. These are primarily implemented to address threats originating from external sources. On the other hand, our research extends the use of Linux qdisc by collaborating with OP-TEE to provide a secure response against cram attacks that originate internally within the system.

The GlobalPlatform Socket API [17], [18] provides a mechanism to prevent data within the socket from being hijacked or lost by establishing a socket in secure world. However, data traversing through this socket also passes through the transmit queues in the normal world, making it susceptible to loss due to cram attacks. We have identified the transmit queue as the initial target point of cram attacks and have devised countermeasures to ensure that data drop does not occur even when a cram attack is underway.

VIII. Conclusion

This study introduces the cram attack, a new cyber threat to automotive Ethernet networks, which can severely disrupt the flow of safety-critical data by garbage packet injections. Also, we present a solution approach to the attack by creating a hidden Band within the Linux transmit queues, named Band-S. Since it is designed to protect essential driving functions, Band-S is structured to be exclusively accessible through OP-TEE based on the ARM TrustZone, ensuring that only authorized applications can transmit data through Band-S. This configuration effectively blocks any attempts to set the TOS for accessing Band-S in the kernel space. Our experiments show that, by utilizing Band-S, the cram attack can be effectively neutralized by providing the safety-critical data flow with a hidden passage that cannot be crammed by garbage packets. This approach ensures the continuity of essential driving functions even amidst attacks, emphasizing the importance of resilient and adaptive vehicular communication systems for uninterrupted vehicle operations.

Acknowledgement

This work was supported partially by Korea Institute for Advancement of Technology (KIAT) grant funded by the Korea Government (MOTIE) (P0017120, The Competency Development Program for Industry Specialist), partially by the National Research Foundation of Korea (NRF) grant funded by the Korean government (MSIT) (2022R1A2C1013197), and partially by Hyundai Motor Company. J.-C. Kim is the corresponding author of this paper.

References

[1] L. Lo Bello, G. Patti, and L. Leonardi, "A perspective on ethernet in automotive communications—current status and future trends," *Applied Sciences*, vol. 13, p. 1278, 2023.

[2] R. S. Rathore, C. Hewage, O. Kaiwartya, and J. Lloret, "In-vehicle communication cyber security: Challenges and solutions," *Sensors*, vol. 22, no. 17, p. 6679, 2022.

[3] D. Kenjić and M. Antić, "Connectivity challenges in automotive solutions," *IEEE Consumer Electronics Magazine*, vol. 12, no. 5, pp. 53–59, 2023.

[4] D. Wang and S. Ganesan, "Automotive network security," in *Proc. 17th International Conference on Electro Information Technology (EIT)*. IEEE, 2021, pp. 193–196.

[5] A. D. Keromytis, "Network bandwidth denial of service (DoS)." pp. 836–838, 2011.

[6] "OP-TEE: Open portable trusted execution environment," https://docs.nvidia.com/jetson/archives/r35.4.1/DeveloperGuide/text/SD/Security/OpTee.html, accessed: 2024-01-30.

[7] M. A. Brown, "Traffic control howto," *Guide to IP Layer Network*, vol. 49, no. 36, 2006.

[8] "ISO/SAE 21434 setting the standard for connected cars' cybersecurity," https://documents.trendmicro.com/assets/white_papers/wp-setting-the-standard-for-connected-cars-cybersecurity.pdf, accessed: 2024-01-29.

[9] "Linux standard base core specification, generic part," https://refspecs.linuxbase.org/LSB_5.0.0/LSB-Core-generic/LSB-Core-generic/baselib-setsockopt-2.html, accessed: 2024-01-30.

[10] U. P. Patel, A. K. Patel, and F. A. Suthar, "The study of digital signature authentication process," *Journal of Information, Knowledge and Research in Computer Science and Applications*, vol. 1, no. 2, 2019.

[11] H. Nishimura, Y. Omori, T. Yamashita, and S. Furukawa, "Secure authentication key sharing between mobile devices based on owner identity," in *Proc. 4th International Conference on Mobile and Secure Services (MobiSecServ)*. IEEE, 2018, pp. 1–6.

[12] "Globalplatform api," https://optee.readthedocs.io/en/latest/architecture/globalplatform_api.html, accessed: 2024-01-29.

[13] Ł. Apiecionek, J. M. Czerniak, and W. T. Dobrosielski, "Quality of services method as a DDoS protection tool," in *Proc. 7th Intelligent Systems*. Springer International Publishing, 2015, pp. 225–234.

[14] A. Garg and A. N. Reddy, "Mitigation of DoS attacks through QoS regulation," *Microprocessors and Microsystems*, vol. 28, no. 10, pp. 521–530, 2004.

[15] F. Benamrane, M. Ali, D. K. Luong, Y.-F. Hu, J.-P. Li, and K. Abdo, "Bandwidth management in avionic networks based on SDN paradigm and ML techniques," in *Proc. 38th Digital Avionics Systems Conference (DASC)*. IEEE, 2019, pp. 1–9.

[16] J. Kim and J.-H. Lee, "Performance evaluation of queueing mechanisms for network-based intrusion prevention system," in *Proc. 11th International Symposium on Soft Computing and Intelligent Systems (SCIS & ISIS)*. Japan Society for Fuzzy Theory and Intelligent Informatics, 2010, pp. 632–637.

[17] "Globalplatform device technology TEE sockets API specification," https://globalplatform.org/wp-content/uploads/2017/01/GPD_TEE-Sockets-API-_v1.0.pdf, accessed: 2024-01-29.

[18] C. Göttel, P. Felber, and V. Schiavoni, "iperftz: Understanding network bottlenecks for trustzone-based trusted applications," *International Symposium on Stabilizing, Safety, and Security of Distributed Systems*, pp. 178–193, 2019.

Authorized licensed use limited to: Pontificia Universidade Catolica do Rio Grande do Sul (PUC/RS). Downloaded on March 26,2026 at 05:06:47 UTC from IEEE Xplore. Restrictions apply.